ANGULAR

KEY CONCEPTS

"Code as Motivation" Series

"Build dynamic, maintainable, and powerful web apps ,the Angular way."

~ AJNAS THAYYIL

1. Components

- The **building blocks** of every Angular application.
- Each component controls a section of the UI (called a view).
- Made up of three parts: **TypeScript** (logic), **HTML** (template), and **CSS** (style).
- Promotes **reusability** and **modular architecture**.

```
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { ToastrService } from 'ngx-toastr';


@Component({
 selector: 'app-products',
 templateUrl: './products.component.html',
 styleUrls: ['./products.component.css']
})
export class ProductsComponent implements OnInit {
```

~ AJNAS THAYYIL

2. Data Binding

- Connects the TypeScript logic with the template (UI).
- Mainly 4 types
  1. **Interpolation – {{ data }}**
  2. **Property binding – [property]="data"**
  3. **Event binding – (event)="handler()"**
  4. **Two-way binding – [(ngModel)]="data"**
- Keeps the UI and data in sync automatically.

```
Two-Way Data Binding

<!-- HTML Template -->
<input [(ngModel)]="userName" placeholder="Enter
your name">
<p>Hello, {{ userName }} </p>

<!-- TypeScript -->
export class GreetingComponent {
  userName = '';
}
```

~ AJNAS THAYYIL

3. Services & Dependency Injection

- Services handle shared logic and reusable data across components.
- Dependency Injection (DI) helps manage and provide instances efficiently.
- Promotes separation of concerns — UI stays clean, logic stays modular.

```
@Injectable({ providedIn: 'root' })
export class MessageService {
  getMessage() { return 'Keep coding confidently '; }
}
```

~ AJNAS THAYYIL

4. Routing

- Manages navigation between pages (components).
- Uses <router-outlet> to display routed content.
- Supports lazy loading, route guards, and parameters for flexibility.

```ts
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';

const routes: Routes = [
  { path: 'home', component: HomeComponent },
  { path: 'about', component: AboutComponent }
];
```

~ AJNAS THAYYIL

5. Directives

- In Angular, Directives are special instructions that you can attach to HTML elements to **add behavior, manipulate the DOM, or modify** the layout dynamically
- Angular has three main types of directives:

1. **Component Directives**

2. **Structural Directives**

*ngIf — conditionally adds/removes elements.

*ngFor — repeats an element for each item in a list.

*ngSwitch — toggles between multiple views.

3. **Attribute Directives**

[ngClass], [ngStyle],

Custom directives you can create yourself

~ AJNAS THAYYIL

6. Pipes

- Transform data directly in the template.
- Built-in pipes: date, uppercase, currency, etc.
- You can also create custom pipes for special formatting.

```
//Date pipe
<p>{{ birthday | date:'longDate' }}</p>
<p>Today: {{ today | date:'fullDate' }}</p>
//Uppercase & Lowercase Pipes
<p>{{ 'angular' | uppercase }}</p>
<p>{{ 'LEARNING IS FUN' | lowercase }}</p>
// Currency Pipe
<p>{{ 2500 | currency:'INR':'symbol':'1.0-0' }}</p>
<p>{{ 99.99 | currency:'USD' }}</p>
// Percent Pipe
<p>{{ 0.875 | percent }}</p>
```

~ AJNAS THAYYIL

7. Lifecycle Hooks

- Angular components go through a lifecycle from creation to destruction.
- Hooks like ngOnInit, ngOnChanges, ngAfterViewInit, etc.
- Help you manage initialization, updates, and cleanup efficiently.

```
ngOnInit() { console.log('Component loaded '); }
ngOnDestroy() { console.log('Component destroyed '); }
```

~ AJNAS THAYYIL

8. Observables & RxJS

- Used for asynchronous operations (HTTP calls, events, streams).
- Part of Reactive Programming using RxJS.
- Core methods: map(), filter(), switchMap(), subscribe().
- Makes Angular apps responsive and reactive.

```
this.data$.subscribe(data => console.log('Received:', data));
```

~ AJNAS THAYYIL

9. Forms

- Two approaches:
  - Template-driven (simpler)
  - Reactive (more control)
- Handle validation, data collection, and submission easily.
- Reactive forms for better for complex, dynamic form logic.

```ts
this.userForm = new FormGroup({
  name: new FormControl('', Validators.required)
});
```

~ AJNAS THAYYIL

10. Modules

- Organize Angular applications into logical units.
- Each module can contain components, directives, and services.
- Helps with lazy loading and scalability.

```
@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

~ AJNAS THAYYIL

"Want More...
get in touch"

~ AJNAS THAYYIL